

Memoria y evaluación de cuestiones teóricos-prácticas

Ejercicio 1)

	Tiempo [ms]	Tamaño de la ruta	Nodos expandidos	Profundidad máxima evaluada	Nodos en abiertos al final de la ejecución	Nodos en cerrados al final de la ejecución	Número de actualizaciones de la heurística
Profundidad - 0	3	66	157	66	-	-	-
Profundidad - 5	3	217	272	217	-	-	-
Profundidad - 6	5	139	166	139	-	-	-
A* - 0	8	48	209	-	5	209	-
A* - 5	31	188	2621	-	51	2621	-
A* - 6	30	78	1980	-	100	1980	-
RTA* - 0	1	38	38	-	-	-	-
RTA* - 5	19	1941	1941	-	-	-	-
RTA* - 6	11	403	403	-	-	-	-
LRTA*(k) - 0	2	38	38	-	-	-	15
LRTA*(k) - 5	39	3203	3203	-	-	-	39
LRTA*(k) - 6	20	343	343	-	-	-	363

En primer lugar, la búsqueda en profundidad se trata de un algoritmo que no usa información heurística, es decir, es búsqueda no informada y es offline, ya que calcula el plan completo antes de comenzar la ejecución de las acciones. Además no es completo en general (puede quedarse en bucles infinitos) y no garantiza encontrar el camino óptimo. Esto último es algo que podemos ver en la tabla, ya que

aunque en estos tres casos si encuentra la solución, necesita bastantes más pasos que el algoritmo A^* , lo que nos indica que no encuentra el camino óptimo. Sin embargo, es mucho más rápido debido a que solo necesita llevar almacenada la información de los nodos visitados, y el camino se recupera siguiendo los punteros a nodos padres desde el objetivo.

En conclusión podemos decir que si nos aseguramos que nuestro problema es completo y no necesitamos obtener la solución óptima y/o hay abundantes soluciones, puede ser una buena aproximación inicial, en caso contrario necesitaríamos algoritmos más complejos. Además tenemos que tener información completa del entorno y este tiene que ser estático, algo que no siempre es posible.

En segundo lugar, el algoritmo A^* sigue siendo offline, pero ahora se usa información heurística para encontrar el camino óptimo. Esto hace que explore muchos menos nodos que algoritmos ciegos. Es importante que la heurística sea buena y adecuada para nuestro problema, ya que de lo contrario puede ser engañosa. Por ejemplo, para el problema *Catapults*, la heurística distancia Manhattan al portal se podría mejorar añadiendo que si todavía no se tiene la llave, la heurística sea la distancia Manhattan a la llave más cercana.

Podemos ver en la tabla que se trata de un algoritmo mucho más costoso, la estructura de abiertos y cerrados para garantizar que se encuentre la solución óptima tiene el inconveniente que va a implicar un mayor uso de la memoria, además de un mayor número de nodos explorados (ya que se va expandiendo en forma de red para asegurarse encontrar el mejor camino) y el consecuente aumento en el tiempo de ejecución. Por lo tanto este algoritmo es recomendable usarlo cuando queremos encontrar el camino más óptimo posible, aunque disponemos de una mayor cantidad de recursos y tiempo para obtenerlo. Al igual que con la búsqueda en profundidad, tenemos que tener información completa del entorno, y este tiene que ser estático, algo que no siempre es posible.

En tercer lugar, el algoritmo RTA^* (Real-Time A^*) se trata de un algoritmo de búsqueda en tiempo real, ya que calcula solo la siguiente acción basándose en un espacio local de búsqueda e incorpora un espacio local de aprendizaje: actualiza la heurística de los estados visitados para escapar de mínimos locales.

Como podemos ver en la tabla, es un algoritmo cuyo tiempo de ejecución va a depender tanto de cómo se van expandiendo los sucesores y de la heurística, ya que al tener información limitada del mapa, una mala decisión va a hacer que se tengan que añadir acciones hasta volver a recuperar la dirección correcta.

Por otro lado, no garantiza el camino óptimo, de hecho puede ni encontrar una solución, como sucede en algunos mapas de la práctica en los que muere el agente a pesar de existir una solución. Sin embargo, al no requerir tener información

completa del entorno lo hace ideal para trabajar en entornos desconocidos o donde solo podemos obtener información limitada a través de por ejemplo unos sensores. Además al calcular una acción, ejecutar, y luego volver a calcular la siguiente, permite usarlo en entornos dinámicos, donde elementos del mapa pueden ir cambiando. Por último también es recomendable usarlo cuando se requiere hacer un movimiento en muy poco tiempo y no se puede esperar a que A^* espere de pensar.

En último lugar, el algoritmo $LRTA^*(k)$ (Learning Real-Time A^*) se trata también de un algoritmo de búsqueda online e incorpora un espacio local de aprendizaje. Es una evolución natural de RTA^* , las únicas diferencias que tiene con el algoritmo RTA^* es por un lado la regla de aprendizaje, ya que RTA^* usa el segundo mínimo y $LRTA^*$ usa el primero, esto hace que aproxime $h^*(n)$ (la heurística que da el coste del camino óptimo desde el nodo n al objetivo) en múltiples ejecuciones, en donde se parte de la h aprendida en la ejecución anterior, de ahí el nombre, ya que pretende aprender a aproximar mejor $h^*(n)$. Por otro lado, la (k) viene de que $LRTA^*(k)$ se trata de una versión parametrizada de $LRTA^*$ en donde se propagan los cambios heurísticos encontrados en hasta k nodos si el cambio proviene de su soporte (mejor vecino). Esto hace que aumente su capacidad detectar mínimos locales antes de meterse de lleno en ellos. Evidentemente mirar k pasos hacia adelante requiere más cálculo por movimiento.

Luego aumenta el rendimiento con respecto a $LRTA^*$ a consta de un mayor tiempo de procesamiento. En la tabla podemos ver que efectivamente se trata de un algoritmo costoso, pero sobre todo cuando se explota la cualidad de actualizar la heurística a los k vecinos, como sucede en gran cantidad de ocasiones en el mapa número 6, conseguimos disminuir el número de nodos visitados, en cambio en el mapa 5 el número de nodos explorados es mucho mayor que en el caso de RTA^* .

En conclusión, al igual que RTA^* , ni garantiza el camino óptimo y puede no encontrar una solución si quiera. Pero es ideal en entornos desconocidos y/o dinámicos y donde se busque rapidez de movimiento añadiendo también un poco de planificación. Además si estamos dispuestos a ejecutar en repetidas ocasiones usando como nuevo valor de la heurística el de la anterior ejecución, iremos convergiendo hacia $h^*(n)$.

Ejercicio 2)

En primer lugar, el estado debería almacenar la posición del avatar, si tiene una de las capas o ninguna, y en caso de tener una el color de la misma. Además se tendría que incluir información dinámica de las capas en el mapa, ya que al coger una esta desaparece del mapa, al igual que al cambiar de capa que se pierde la que llevas puesta y desaparece la que coges.

Por otro lado, la búsqueda bidireccional puede plantear las siguientes problemáticas: En la parte forward, partes de la salida, con ninguna capa puesta y sabes donde están las capas en el mapa. En cambio, si partes desde el objetivo, ya no sabes con qué capa llegaste al objetivo, o directamente si llegaste con alguna capa, luego vas a tener que ir ramificando las posibles soluciones, y cuantos más elementos dinámicos incluyas (las capas en este caso), más complejo va a ser.

Algo parecido ocurre cuando se encuentran la parte forward y backward, necesario para que la búsqueda tenga éxito (en ese nodo compartido se une el camino de avance y retroceso para formar la solución completa), ya que deben de coincidir evidentemente en la misma coordenada del mapa, pero además el resto de elementos del estado deben coincidir, es decir, deben llevar la misma capa puesta o ninguna, y deben de quedar el mismo número de capas en el mapa, y debido a que la búsqueda backward tiene que explorar todas las combinaciones de los elementos dinámicos, esto hace que se consuman muchos recursos de memoria y tiempo.

Por esto mismo puede que el camino encontrado no sea óptimo, ya que por ejemplo la búsqueda forward puede llegar a una casilla X con la capa azul y habiendo recogido ya la roja que ya desapareció, y en cambio la backward puede llegar a la misma casilla X, con la capa azul, pero suponiendo que la capa roja sigue existiendo, luego si se une ese camino, no se podría ejecutar.

En conclusión, en juegos con elementos dinámicos y estados irreversibles que modifican el entorno, la búsqueda bidireccional se enfrenta a una serie de problemáticas que la hacen altamente ineficiente. En cambio, en este caso, un algoritmo como A* es mucho más eficiente y fácil de implementar gracias a que va solo en una dirección y se trata de una búsqueda informada.

Ejercicio 3)

- a) Si el agente empieza en una posición diferente, no hay problema ya que las heurísticas las va aprendiendo para todas las casillas, y el objetivo sigue siendo el mismo, luego la heurística desde una casilla sigue siendo la misma aunque el agente se haya desplazado. Lo peor que puede pasar es que el agente empiece en una zona que todavía no ha explorado, y tenga que usar la heurística inicial para guiarse.
- b) En este caso, la situación si cambia, ya que las heurísticas aprendidas ya no nos sirven, debido a que el objetivo es otro. Las heurísticas anteriores tienen como objetivo ayudarnos a encontrar el camino al objetivo anterior, pero si este es un estado diferente, las heurísticas que teníamos guardadas solo nos van a entorpecer. Además en caso de no modificar la heurística, el agente va a tender a ir al anterior objetivo, del que le va a costar mucho poder salir.

Ejercicio 4)

```

wwwwwwwwwwwwwwwwww
wA_____w
w_____w
wC_____w
w_____w
w1_____w
w_____w
w_____w
w_____w
wk_____w
w_____w
w_____w
w_____w
w_____w
wg_____w
wwwwwwwwwwwwwwwwww

```

Figura 1: Captura de pantalla de “mapaejercicio4.txt”

He elegido como elemento del estado el número de monedas. En este juego, las monedas afectan a las acciones posibles, ya que para usar una catapulta el avatar debe tener al menos una moneda y, además, al utilizarla la gasta.

En este mapa hay una misma casilla, la situada justo encima de la catapulta, a la que se puede llegar de dos maneras distintas: una pasando antes por la moneda y otra rodeando por la derecha sin recogerla. Aunque la posición del avatar sea la misma, no se trata del mismo estado, porque en un caso el agente llega con moneda y en el otro sin ella.

A partir de esa casilla, las acciones posibles cambian. Si el avatar tiene moneda, puede usar la catapulta y tomar un atajo hacia la llave y la meta. Si no la tiene, no puede usar la catapulta y debe continuar por el camino largo.

Por tanto, si el número de monedas no se incluyera en el estado, el algoritmo trataría esas dos situaciones como si fueran iguales cuando en realidad no lo son. Eso podría hacer que confundiera caminos distintos o que encontrara una solución no óptima.